



1ST YEAR BASIC TRAINING IN CYBER SECURITY



INTRODUCTION TO OPERATING SYSTEMS 1 (SYST1)

Dr. Sassi BENTRAD

✉ : sassi.bentrad.enscs@gmail.com || sassi.bentrad@enscs.edu.dz

LISCO Laboratory (Laboratoire d'Ingénierie des Systèmes Complexes) / University of Badji Mokhtar-Annaba (UBMA)
National School of Cyber Security (NSCS)

Basic Training in Cyber Security (1BT)
Formation de Base en Cyber-Sécurité (1FB)



CHAPTER

7

BASH PROGRAMMING AND SHELL SCRIPTING

SYST1'2025/2026

BASH PROGRAMMING AND SHELL SCRIPTING (10 %)



**COURSE
CONTENT**

CHAPTER 7

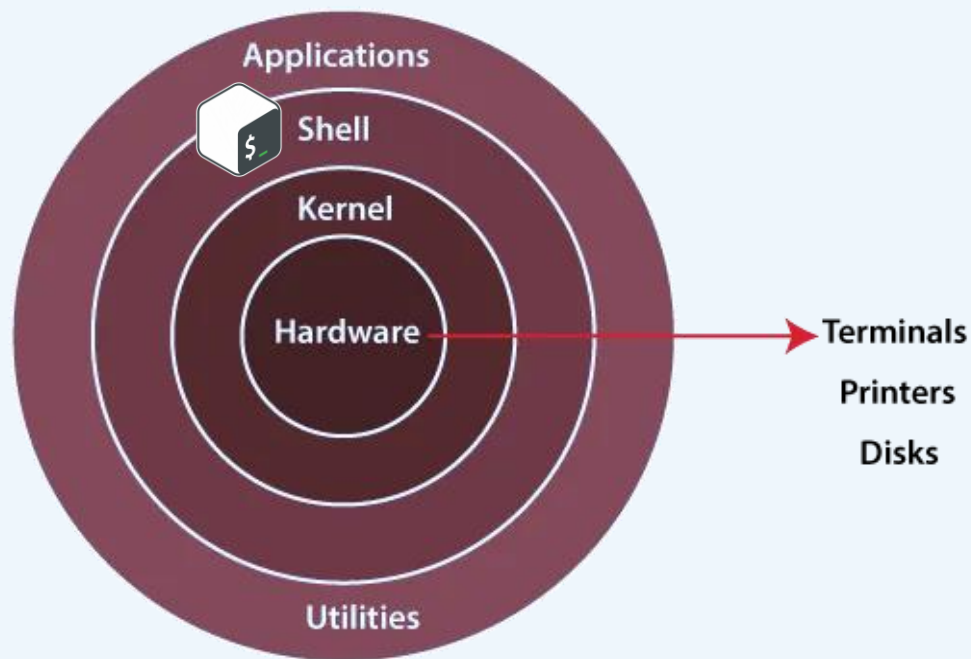
- ☐ Introduction
- ☐ Bash Programming vs. Shell Scripting
- ☐ Shell Scripting Basics
- ☐ Shell Programming
- ☐ Summary
- ☐ Reference Books

❑ Linux Architecture

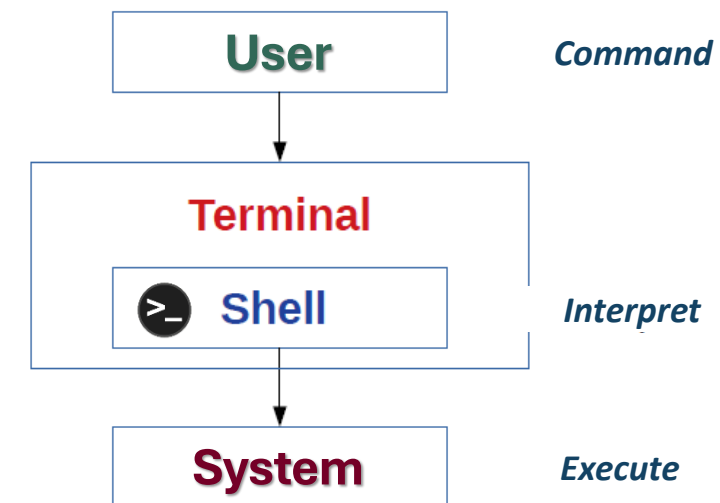
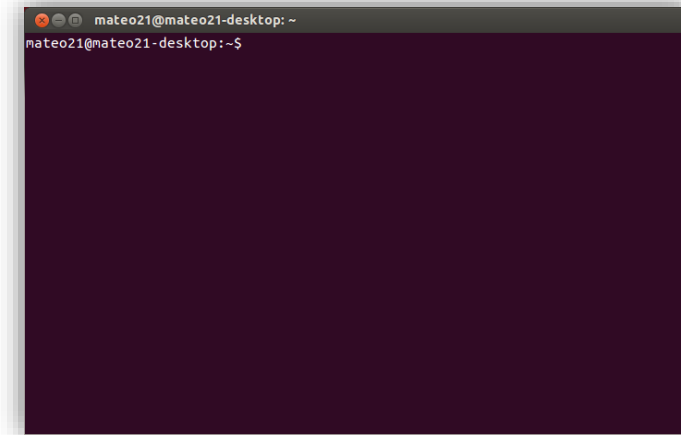
Some **Linux distributions** install several shells.

These might include :

gnome-terminal, konsole, xterm, rxvt, kvt, nxterm, and eterm.



Architecture of Linux System



Command Execution Process

❑ What is a “Shell”?

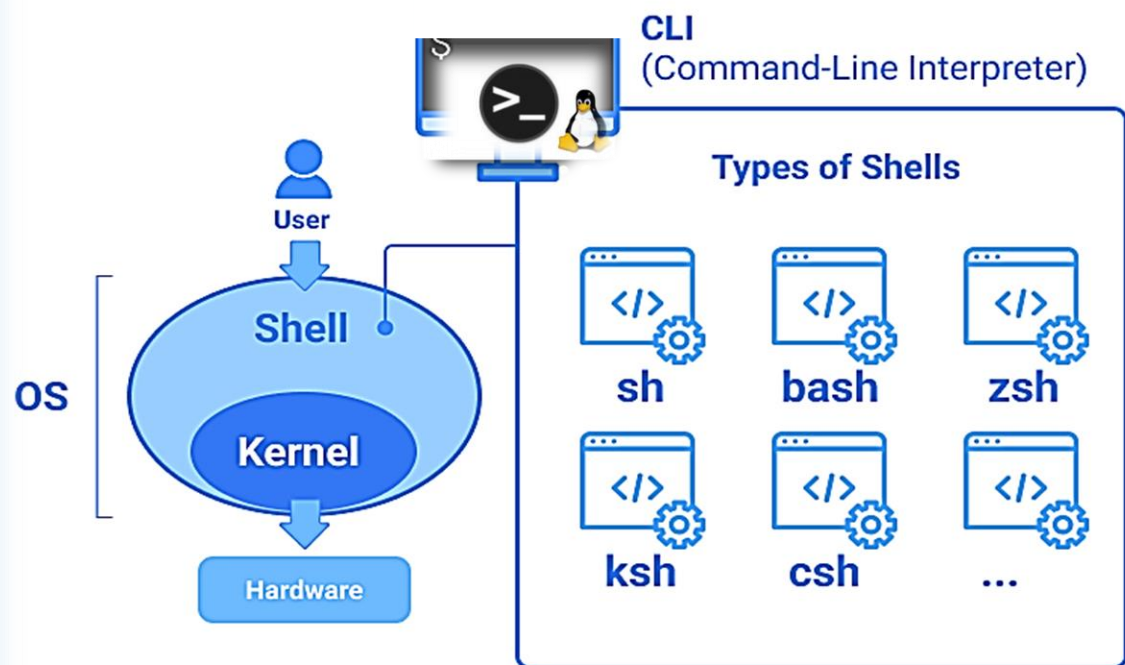
The “**Shell**” is simply *another program* on top of the kernel which provides a **basic human-OS interface**.

- It is a command interpreter
 - ✓ Built on top of the **kernel**
 - ✓ Enables users to run services provided by the **UNIX/Linux OS**
- In its simplest form, a series of commands in a file is a shell program that saves having to retype commands to perform common tasks.

How to know what shell you use ? `echo $SHELL`



Most famous and popular shells



Shells ...

- ▶ Bourne Again Shell (bash)
- ▶ Z Shell (zsh)
- ▶ Bourne Shell (sh)
- ▶ C Shell (csh)
- ▶ TENEX C Shell (tcsh)
- ▶ KornShell (ksh)
- ▶ Debian Almquist Shell (dash)
- ▶ Friendly Interactive Shell (fish)



❑ Which Shell to Use ?

- **sh** (Bourne shell) was considered better for programming
- **csch** (C-Shell) was considered better for interactive work.
- **tcsh** and **korn** were improvements on c-shell and bourne shell respectively.
- **bash** (Bourne Again Shell) is largely compatible with **sh** and also has many of the nice features of the other shells
- On many systems such as our LINUX clusters **sh** is symbolically linked to bash, **/bin/sh -> /bin/bash**
- We recommend that you use **sh/bash** for writing new shell scripts but learn **csch/tcsh** to understand existing scripts.
- Many, if not all, scientific applications require **csch/tcsh** environment (GUI, Graphics Utility Interface)
- All **Linux versions** use the **Bash shell** as the default shell
 - ✓ *Bash / Bourn / ksh / sh prompt: \$*
- All **UNIX system** include **C shell** and its predecessor **Bourne shell**.
 - ✓ *Csh / tcsh prompt: %*

❑ Linux Shell

To check shell:

```
$ echo $SHELL (shell is a pre-defined variable)
```

To switch shell:

```
$ exec shellname (e.g., $ exec bash or simply type $ bash)
```

- ✓ You can switch from one shell to another by just typing the name of the shell.
- ✓ **exit** return you back to previous shell.

❖ INTRODUCTION

In today's computing world, **automation is key**, and **mastering shell scripting is an essential skill** for system administrators, developers, and cybersecurity professionals. The **Bash shell** (*Bourne Again Shell*) is one of the most widely used command-line interfaces in Linux and Unix-based systems, making it a powerful tool for system management and process automation.

Throughout this course, we will explore how to:

- ✅ Write and execute Bash scripts to automate repetitive tasks.
- ✅ Understand shell syntax, variables, loops, and conditional statements.
- ✅ Work with system commands, pipelines, and redirections.
- ✅ Manage files, processes, and system resources efficiently.
- ✅ Develop robust scripts with error handling and debugging techniques.

➡ *The **first step** to hands-on experience and real-world applications*



INTRODUCTION

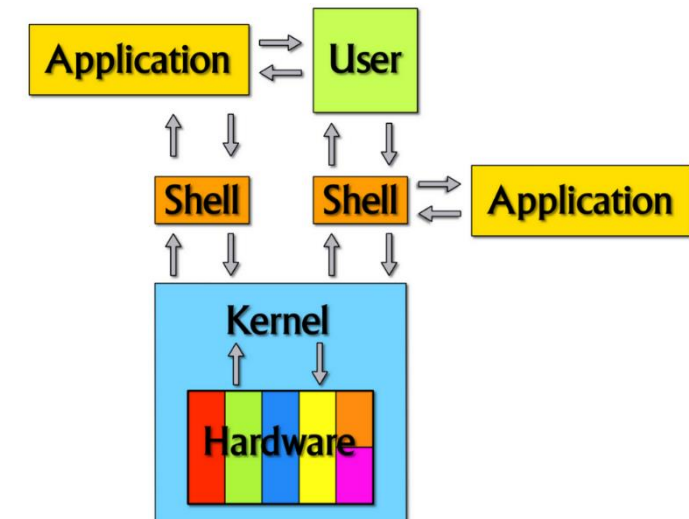
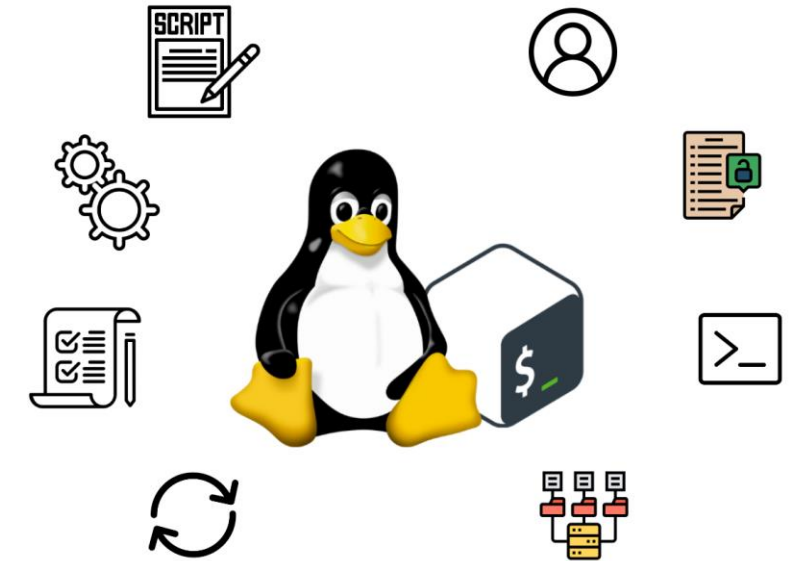
□ Bash Programming vs. Shell Scripting

- **Bash programming** refers specifically to writing programs or scripts using **Bash**, the **Bourne Again Shell**, which is a type of shell (command-line interpreter).

Bash programming focuses on creating *logic and control flows* (like loops, conditionals, functions) within the context of the Bash shell environment.

- **Shell scripting** is a broader concept that involves writing scripts for **any shell** (not just Bash).

Shells include **Bash, Zsh, Fish, Ksh, and others**. A **shell script** is essentially a *sequence of commands* written for a shell to automate tasks or solve problems.



❑ Bash Programming vs. Shell Scripting : Key Differences

Aspect	Shell Script (POSIX sh)	Bash Script
Shell Used	Could be for any shell (Bash, Zsh, sh, etc.)	Specifically for Bash
Shell Compatibility	Works with any POSIX-compliant shell (e.g., sh, dash, ksh).	Requires the Bash shell (bash).
Features	Limited to POSIX-standard syntax and features.	Supports advanced features like arrays, associative arrays, [[]] tests, =~ regex, process substitution, etc.
Portability	More portable across Unix/Linux systems.	Less portable (Bash-specific features may not work in other shells)
Shebang Line	#!/bin/sh	#!/bin/bash or #!/usr/bin/env bash
Syntax	Simpler, stricter syntax. Depends on the shell being used.	Uses Bash-specific syntax and features. More flexible syntax with shortcuts
Advanced Features	Basic control structures, functions, etc.	Supports arrays, [[...]] for conditionals, string manipulation, etc.

❑ Bash Programming vs. Shell Scripting : Key Differences

When to use Which ?

■ Use POSIX Shell Script (sh):

- For scripts that need to run on **any Unix/Linux system** (e.g., embedded systems, older servers).
- Example: *Scripts in /bin or system initialization scripts (e.g., those in /etc/init.d).*

■ Use Bash Script:

- For **Linux-centric scripts** where advanced features are needed (e.g., arrays, string manipulation).
- Example: *Complex automation tasks, scripts leveraging Bash-specific syntax.*

sh

```
#!/bin/sh
name="World"
echo "Hello, $name"
for i in 1 2 3; do
    echo "$i"
done
```

bash

```
#!/bin/bash
name="World"
echo "Hello, $name"
# Bash-specific features:
for i in {1..3}; do # Brace expansion
    echo "$i"
done
# Using an array:
files=(*.txt)
echo "Text files: ${files[@]}"
```

❑ Bash Programming vs. Shell Scripting : Summary

- **Shell script** = Broad term for scripts compatible with **any shell**.
- **Bash script** = A subset of shell scripts using **Bash-specific features** (*A specific type of shell script written for the **Bash** shell, often using its unique features*).
- **Portability**: Use **POSIX sh** for maximum compatibility; use **Bash** for richer functionality on Linux.
- **Always check** the **shebang line** (`#!/bin/sh` vs. `#!/bin/bash`) and **test your script in the target environment!**

❖ INTRODUCTION

❑ Objectives

- Understand the basics of Shell programming and scripting
- Learn how to create and execute Shell scripts
- Explore common use cases for automation and task management
- Understand key Shell commands and syntax for effective scripting
- Gain hands-on experience through practical examples and exercises.

❑ Prerequisites

- Basic knowledge of Linux command-line usage
- Familiarity with file system navigation and permissions
- A working knowledge of text editors (e.g., Vim, Nano)
- Basic programming concepts (optional but helpful)

❖ INTRODUCTION

❑ Tools and Environment Setup



- Installing and configuring Bash Shell (or other relevant Shells)
- Installing Bash (**Linux, Windows WSL, Cygwin, Git Bash**)
- Recommended text editors and IDEs for writing scripts (**vim, nano, VS Code**)
- Basic configuration of environment variables
- Understanding Shell Configuration Files (.bashrc, .bash_profile)
- Debugging tools (set -x, shellcheck)



❑ How to get the most out of this chapter

- Hands-on practice approach: practicing by writing scripts
- Experimenting with small scripts before building complex ones
- Following best practices and coding standards
- Understanding the importance of experimentation and debugging
- Leveraging online resources and communities for further learning
- Tips for breaking down and solving problems efficiently

❑ Real-World Applications

- Automating repetitive tasks (file management, backups, etc.)
- Automating system administration tasks (user management, process monitoring)
- Data processing and parsing text files
- Scheduling tasks with cron jobs, Log file analysis and monitoring
- Server management and DevOps automation
- Security scripting and penetration testing
- Use in cloud computing, virtual machines, and containerization

❑ Importance of Bash Programming and Shell Scripting

- Efficiency and productivity benefits in system administration and development
- Essential for DevOps, system administration, and cybersecurity
- Enhancing productivity by reducing manual tasks
- Writing portable, lightweight scripts for cross-platform tasks
- Shell scripting as an indispensable tool for power users and IT professionals
- Integration with other programming languages (Python, Perl)



❑ Bash Programming Basics

- **Bash programming** refers specifically to writing programs or scripts using **Bash**, the **Bourne Again Shell**, which is a type of shell (command-line interpreter).
- **Bash programming** focuses on creating logic and control flows (like loops, conditionals, functions) within the context of the Bash shell environment.
- **Scope:** It includes understanding the core features of Bash, such as:
 - ✓ Variables and data types
 - ✓ Functions and control structures (if/else, loops)
 - ✓ Handling input/output
 - ✓ Regular expressions and string manipulation
 - ✓ Process management (background jobs, pipelines, job control)
 - ✓ Advanced scripting techniques, including debugging, error handling, and optimizing code

❑ Bash Programming Basics



▪ Use Case:

Bash programming is used when you want to write more complex, reusable scripts or tools within the Bash environment. It can involve building small utilities, automating repetitive tasks, or interacting with the operating system.

▪ Nature:

More focused on programming logic, structure, and writing code that interacts directly with the operating system via the Bash shell. It often involves writing small, standalone programs or scripts.



❑ Shell Scripting Basics

- **Shell scripting** is a broader concept that involves writing scripts for any shell (**not just Bash**). Shells include **Bash**, **Zsh**, **Fish**, **Ksh**, and others.
- A **shell script** is essentially a sequence of commands written for a shell to automate tasks or solve problems.
- **Scope:** While **Bash** is a specific shell, **shell scripting** can be done in any shell, meaning:
 - ✓ Writing commands to automate tasks in the terminal (e.g., file manipulation, process management)
 - ✓ Making use of shell-specific features and commands for a variety of systems (Linux, macOS, etc.)
 - ✓ Shell scripting can include tools and utilities from the broader Unix/Linux ecosystem, such as grep, awk, sed, and cut.



❑ Shell Scripting Basics

▪ Use Case:

Shell scripting is often used for simpler tasks like automating system maintenance (e.g., backups), running batch jobs, or performing regular administrative actions. It can be used across multiple shell environments, allowing for portability if written in a standard **POSIX**-compliant way.

▪ Nature:

Shell scripting is more about **command-line instructions** that execute in sequence to automate tasks, with less emphasis on complex programming logic (although it can still involve some).

❖ SHELL SCRIPTING BASICS

❑ Shell Script

A **shell script** is a text file that contains a sequence of commands for a Unix/Linux operating system.

In the command line, you can usually run only a single command.

Using the **shell script**, you can **run multiple commands as one operation** without another human intervention during the process.

Each line of a file is treated as a command line.

Two key ingredients :

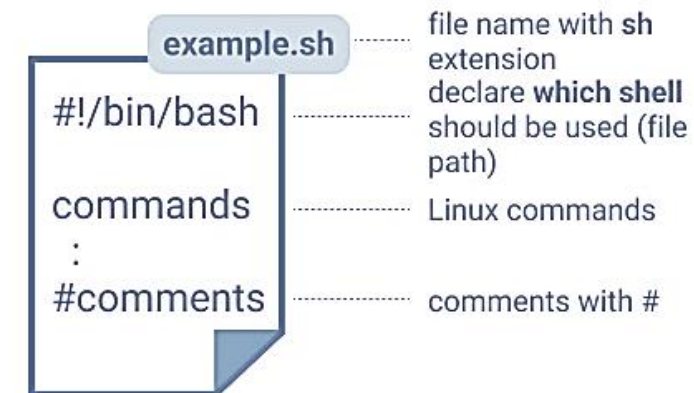
- UNIX/LINUX commands
- Shell programming syntax

Shell Script

A text file that contains a sequence of commands for a Linux or Unix operating system



Document Structure



Access Mode

Add execution permission for the document

```
chmod u+x example.sh
```

Run Script

```
$ file path (e.g., ./example.sh)
```



#!/ Shebang. Indicates which interpreter a script should be run with.

❑ Shell Script

How to develop a shell script?

There are some rules to develop shell scripts.

File name :

Add **.sh** at the end of the file name as a file extension.

Document structure:

- *Declaration:* You need to specify which shell will be used for the shell script.
- *Comment:* You can add comments with **#** at the beginning of comments.

Here is an example of creating a shell script using some commands. Use **vim** to create a shell script file named **test.sh**.

Command Line- INPUT

```
ubuntu $ | vim test.sh
```

test.sh

```
#!/bin/bash

# display product items
cat purchase_product.txt

# extract unique items and count duplicated items
sort purchase_product.txt | uniq -c
```

To successfully write a **shell script**, we have to do **3 things**:

1. Write a script
2. Give the shell permission to execute it
3. Put it somewhere the shell can find it

❑ Shell Script

Add an execution permission :

Usually, execution permission is not added when you create a file. In order to make the script executable, you need to add an execution permission to the file. To grant permission to the owner of the file, run the **chmod u+x** command followed by the file path.

For the **test.sh** file, grant the execution permission to the owner user.

Command Line- INPUT

```
ubuntu $ | sudo chmod u+x test.sh
```

Run a shell script :

To run a shell script, you just need to specify the file path of the shell script like the one below.

Command Line- INPUT

```
ubuntu $ | ./test.sh
```

You'll see the original list of purchased products followed by unique product items with numbers of duplicated items.

To make a script executable :

Syntax: **chmod u+x script_name.sh**

To run it :

Syntax: **./script_name.sh**

To avoid **./**

Syntax:

PATH+=:.

or

PATH=\$PATH:.

❖ SHELL SCRIPTING BASICS

❑ Shell Script

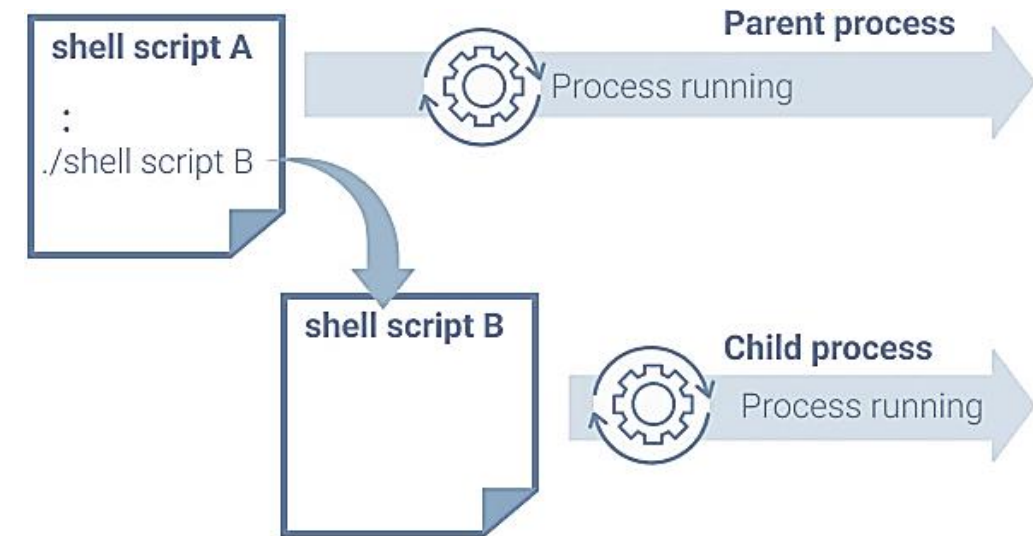
Call another shell script from a shell script

You can call another shell script from a shell script. This is like creating a child process from a parent process.

To call another shell script, you just need to add a shell script execution command to the shell script.

For example, create a new shell script file named *test_child.sh* and edit the file as shown below.

Also, copy the *test.sh* file and create a new file *test_parent.sh*. Then, edit the file to add a command to execute *test_child.sh*.



Command Line- INPUT

```
ubuntu $ | vim test_child.sh
```

>test_child.sh

```
#!/bin/bash
echo "-----"
echo "Unique Product List with # of Duplicated Items"
```

Command Line- INPUT

```
ubuntu $ | cp test.sh test_parent.sh
ubuntu $ | vim test_parent.sh
```

❖ SHELL SCRIPTING BASICS

❑ Shell Script

Call another shell script from a shell script

Add ***./test_child.sh*** between the **cat** command and the **sort** command.

To execute the shell scripts, add permissions and run the ***test_parent.sh*** like shown below.

You can see that ***test_parent.sh*** successfully called ***test_child.sh***.

test_parent.sh

```
#!/bin/bash

# display product items
cat purchase_product.txt

./test_child.sh

# extract unique items and count duplicated items
sort purchase_product.txt | uniq -c
```

Command Line- INPUT

```
ubuntu $ | sudo chmod u+x test_parent.sh
ubuntu $ | sudo chmod u+x test_child.sh
ubuntu # | ./test_parent.sh
```

❖ SHELL SCRIPTING BASICS

❑ UNIX/Linux Commands



- File Management and Viewing
- File System Management
- Help, Job/Process Management
- Network Management
- System Management
- User Management
- Printing and Programming
- Document Preparation
- Miscellaneous

- To understand the working of the command and possible options use (**man** command)
- Using the GNU Info System (**info**, info command)
- Listing a Description of a Program (**whatis** command)
- Many tools have a long-style option, `**--help**', that outputs usage information about the tool, including the options and arguments the tool takes.



❑ Useful Commands in Scripting

■ **grep**

- Pattern searching
- Example: **grep** 'boo' filename

■ **sed**

- Text editing
- Example: **sed** 's/XYZ/xyz/g' filename

■ **awk**

- Pattern scanning and processing
- Example: **awk** '{print \$4, \$7}' filename



❖ SHELL SCRIPTING BASICS

❑ My First Shell Script

- Start **vi** `scriptfilename.sh` with the line
#!/bin/sh
- All other lines starting with **#** are comments
(*make code readable by including comments*).
- Tell the OS that the script file is executable
\$ chmod u+x scriptfilename.sh
\$ chmod +x scriptfilename.sh
- Execute the shell-script
\$./scriptfilename.sh

```
$ vi myfirstscript.sh

#!/bin/sh
# The first example of a shell script
directory=`pwd`
echo Hello World!
echo The date today is `date`
echo The current directory is $directory

$ chmod +x myfirstscript.sh

$ ./myfirstscript.sh

Hello World!

The date today is Mon Mar 8 15:20:09 EST 2010

The current directory is /netscr/shubin/test
```



❑ Shell Scripts

- **No compiler required** to run a shell script, because the shell acts as an **interpreter** when reading script files.
- After you create a shell script, you simply tell the OS that the file is a program that can be executed, by using the **chmod** command to change the files' mode to be executable.
- Shell programs run **less quickly** than compiled programs, because the shell must interpret each command inside the executable script file before it is executed.

❑ Commenting

- Lines starting with **#** are comments except the very first line where **#!** indicates the location of the shell that will be run to execute the script.
- On any line characters following an unquoted **#** are considered to be comments and ignored.
- **Comments are used to :**
 - ✓ Identify who wrote it and when
 - ✓ Identify input variables
 - ✓ Make code easy to read
 - ✓ Explain complex code sections
 - ✓ Version control tracking
 - ✓ Record modifications



❖ SHELL SCRIPTING BASICS

❑ Quote Characters

There are three different quote characters with different behaviour. These are:

- “ : **double quote**, weak quote. If a string is enclosed in “ ” the references to variables (i.e. `$variable`) are replaced by their values. Also back-quote and escape `\` characters are treated specially.
- ‘ : **single quote**, strong quote. Everything inside single quotes are taken literally, nothing is treated as special.
- ` : **back quote**. A string enclosed as such is treated as a command and the shell attempts to execute it. If the execution is successful the primary output from the command replaces the string.

Example: `echo "Today is:" `date``

❑ Echo

Echo command is well appreciated when trying to debug scripts.

Syntax: `echo {options} string`

Options:

- e : expand `\` (back-slash) special characters
- n : do not output a new-line at the end.

String can be a “weakly quoted” or a ‘strongly quoted’ string.

In the weakly quoted strings the references to variables are replaced by the value of those variables before the output.

As well as the variables some special **backslash_escaped** symbols are expanded during the output. If such expansions are required the `-e` option must be used.



❑ User Input During Shell Script Execution

- As shown on the hello script input from the standard input location is done via the read command.

- **Example:**

```
echo "Please enter three filenames:"  
read filea fileb filec  
echo "These files are used:$filea $fileb $filec"
```

- Each read statement reads an entire line. In the above example if there are less than 3 items in the response the trailing variables will be set to blank ' '.
- Three items are separated by one space.
- **Hello script exercise continued...**

- The following script asks the user to enter his name and displays a personalised hello.

```
#!/bin/sh  
echo "Who am I talking to?"  
read user_name  
echo "Hello $user_name"
```

- Try replacing " with ' in the last line to see what happens.



❖ SHELL SCRIPTING BASICS

❑ Debugging your shell scripts

Debugging shell scripts is essential to identify and fix errors, ensuring smooth execution. Here are some **common techniques and tools for debugging** (we can efficiently troubleshoot shell scripts and improve their reliability):

1. Using the `-x` Option (Trace Mode)

Run the script with `bash -x script.sh` or add `set -x` at the beginning of the script to display each command before execution.

- Generous use of the `echo` command will help.
- Run script with the `-x` parameter.
e.g. `sh -x ./myscript`
or `set -o xtrace` before running the script.
- These options can be added to the first line of the script where the shell is defined.
e.g. `#!/bin/sh -xv`

```
1  #!/bin/bash
2  set -x  # Enable debugging
3
4  echo "Hello, World!"
```



❖ SHELL SCRIPTING BASICS

❑ Debugging your shell scripts

Debugging shell scripts is essential to identify and fix errors, ensuring smooth execution. Here are some **common techniques and tools for debugging** (we can efficiently troubleshoot shell scripts and improve their reliability):

2. Using the **-e** Option (Exit on Error)

Adding **set -e** forces the script to exit immediately if a command fails.

```
1  #!/bin/bash
2  set -e  # Stop execution on error
3
4  mkdir /protected_folder  # Will stop execution if it fails
5  echo "This line will not execute if the previous command fails."
```

3. Using **-v** (Verbose Mode)

The **-v** option prints each command before execution, useful for tracking script execution flow.

```
1  #!/bin/bash
2  set -v  # Enable verbose mode
3
4  echo "This is a test script."
5  ls -l
```



❑ Debugging your shell scripts

Debugging shell scripts is essential to identify and fix errors, ensuring smooth execution. Here are some **common techniques and tools for debugging** (we can efficiently troubleshoot shell scripts and improve their reliability):

4. Using **echo** or **printf** for Debugging

Adding **echo** statements helps trace variable values and script flow.

```
1  #!/bin/bash
2  name="Alice"
3  echo "The name variable is set to: $name"
```

5. Using the **trap** Command

trap helps catch errors and display debugging information before the script exits.

```
1  #!/bin/bash
2  trap 'echo "Error on line $LINENO"' ERR
3
4  echo "Running script..."
5  ls /non_existent_directory # This will trigger the trap
```



❑ Debugging your shell scripts

Debugging shell scripts is essential to identify and fix errors, ensuring smooth execution. Here are some **common techniques and tools for debugging** (we can efficiently troubleshoot shell scripts and improve their reliability):

6. Redirecting Error Messages

Redirect error output to a file for analysis.

```
1  #!/bin/bash
2  ./my_script.sh 2> error_log.txt
```

7. Using shellcheck (Static Analysis Tool)

Install and use **shellcheck** to detect syntax errors and bad practices.

```
1  shellcheck my_script.sh
2
```



❑ Programming features of the UNIX/Linux shell

① Shell variables:

Your scripts often need to keep values in memory for later use. Shell variables are symbolic names that can access values stored in memory.

② Operators:

Shell scripts support many operators, including those for performing mathematical operations.

③ Logic structures:

Shell scripts support **sequential logic** (for performing a series of commands), **decision logic** (for branching from one point in a script to another), **looping logic** (for repeating a command several times), and **case logic** (for choosing an action from several possible alternatives).



❑ Shell Variables

- Variables are symbolic names that represent values stored in memory

- **Three different types of variables :**

- ① **Global Variables:** Environment and configuration variables, capitalized, such as **HOME**, **PATH**, **SHELL**, **USERNAME**, and **PWD**.

When you login, there will be a large number of global System variables that are already defined. These can be freely referenced and used in your shell scripts.

- ② **Local Variables**

Within a shell script, you can create as many new variables as needed. Any variable created in this manner remains in existence only within that shell.

- ③ **Special Variables**

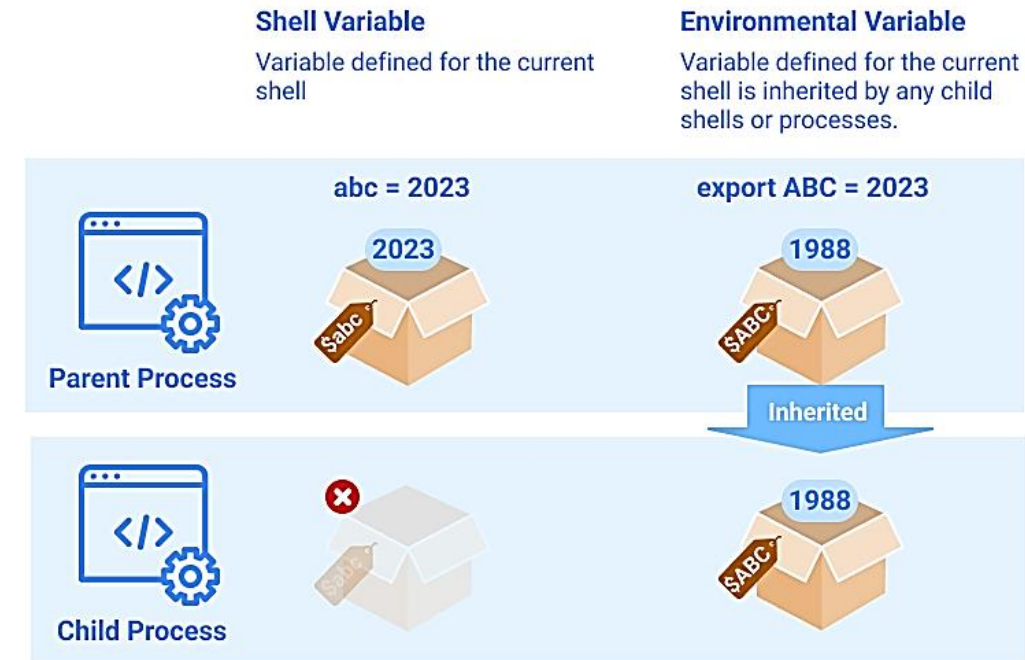
Reversed for OS, shell programming, etc. such as positional parameters **\$0**, **\$1** ...



❖ SHELL SCRIPTING BASICS

❑ Shell Variables / Global Variables (Environment and configuration variables)

- **Environment variables** are **dynamic values** which affect the processes or programs on a computer. They exist in every operating system, but types may vary.
- **Environment variables** can be created, edited, saved, and deleted and give information about the system behavior.
- **Environment variables** can change the way a software/programs behave.



```
[shell variable] = [assigned string]
```

```
export [environmental variable] = [assigned string]
```


❖ SHELL SCRIPTING BASICS

❑ Shell Variables / *some common global variables*

Variable	Description
SHELL	Current Shell being used by the user
USER	The username (Your login name)
HOME	Default path to the user's home directory
EDITOR	Path to the program which edits the content of files
UID	User's unique ID
TERM	Default terminal emulator
PWD	Current working directory
DISPLAY	Used by X-Windows system to identify the display
PATH	This variable contains a colon (:) -separated list of directories in which your system looks for executable files. When you enter a command on terminal, the shell looks for the command in different directories mentioned in the \$PATH variable. If the command is found, it executes. Otherwise, it returns with an error 'command not found'. <pre>guru99@VirtualBox:~\$ echo \$PATH /usr/lib/lightdm/lightdm:/usr/local/sbin:/usr/local/bin:/usr :/bin:/usr/games</pre>

❖ SHELL SCRIPTING BASICS

❑ Shell Variables / Set new environment variables

You can create your own user defined variable, with syntax :

VARIABLE_NAME=variable_value

Again, bear in mind **that variables are case-sensitive** and usually they are created in upper case.

❑ Shell Variables / Deleting Variables

The following syntax can be used to remove a Variable from the system.

unset VARIABLE_NAME

This **would remove the Variable** and its value permanently.

Define a new variable

```
ubuntu@ubuntu:~$ NEWVARIABLE=value1234
```

Check value of the newly created variable

```
ubuntu@ubuntu:~$ echo $NEWVARIABLE
value1234
```

TIP: Do not leave space between variable name and "=" sign
you will get an error

```
ubuntu@ubuntu:~$ NEWVARIABLE = value1234
NEWVARIABLE: command not found
```

TIP: Do not forget the "\$" sign when you check the value of variable

```
ubuntu@ubuntu:~$ echo NEWVARIABLE
NEWVARIABLE
```

Define a new variable

```
guru99@VirtualBox:~$ VARIABLE1=1234
```

Check value of the newly created variable

```
guru99@VirtualBox:~$ echo $VARIABLE1
1234
```

Use the 'unset' command

```
guru99@VirtualBox:~$ unset VARIABLE1
```

The variable value is removed

```
guru99@VirtualBox:~$ echo $VARIABLE1
guru99@VirtualBox:~$
```

❖ SHELL SCRIPTING BASICS

❑ Shell Variables / Accessing variable values

In order to determine value of a variable, use the command

```
echo $VARIABLE
```

Variables are **Case Sensitive**. Make sure that you type the variable name in the right letter case otherwise you may not get the desired results.

The '**env**' command displays **all the environment variables**.

To see a list of your environment variables:

```
$ printenv or: $ printenv | more
```

```
home@VirtualBox:~$ echo $USER
home
home@VirtualBox:~$ echo $HOME
/home/home
home@VirtualBox:~$ echo $UID
1000
home@VirtualBox:~$ echo $TERM
xterm
home@VirtualBox:~$ echo $PATH
/usr/lib/lightdm/lightdm:/usr/local/sbin:/usr/local/bin:/bin:/usr/games
home@VirtualBox:~$
```

```
guru99@VirtualBox:~$ env
SSH_AGENT_PID=8193
DBUS_STARTER_ADDRESS=unix:abstract=/tmp/dbus-3cJBjUA4bT
1018600004deb
GPG_AGENT_INFO=/tmp/keyring-lP9SZ1/gpg:0:1
TERM=xterm
SHELL=/bin/bash
XDG_SESSION_COOKIE=a5fb982bea8a8cb0299c2c9f00000007-134
WINDOWID=58720261
```

❑ Shell Variables / *Defining local variables*

- As in any other programming language, variables can be defined and used in shell scripts.
- Unlike other programming languages, **variables in Shell Scripts are not typed.**
- **Examples :**

a=1234 # a is NOT an integer, a string instead

b=\$a+1 # will not perform arithmetic but be the string '1234+1'

b=`expr \$a + 1` # will perform arithmetic so b is 1235 now.

***Note :** +, -, /, *, **, % operators are available.*

b=abcde # b is string

b=`abcde` # same as above but much safer.

b=abc def # will not work unless 'quoted'

b=`abc def` # i.e. this will work.

***Important note:** do not leave spaces around the =*



❖ SHELL SCRIPTING BASICS

❑ **Shell Variables / Referencing variables** --curly bracket

- Having defined a variable, its contents can be referenced by the **\$** symbol.
 - E.g. **\${variable}** or simply **\$variable**. When ambiguity exists **\$variable** will not work.
 - Use **\${ }** the rigorous form to be on the safe side.

■ **Example:**

```
a= 'abc'
```

```
b=${a}def
```

this would not have worked without the { } as
#it would try to access a variable named adef



❖ SHELL SCRIPTING BASICS

❑ Shell Variables / *Variable List/Array*

- To create lists (array) – round bracket
\$ **set** **Y** = (**INFO** **123** **CS251**)
- To set a list element – square bracket
\$ **set** **Y**[**2**] = **NSCS**
- To view a list element:
\$ **echo** **\$Y**[**2**]

■ Example :

```
#!/bin/sh
a=(1 2 3)
echo ${a[*]}
echo ${a[0]}
```

Results : 1 2 3
1



❑ Shell Variables / Positional Parameters

- When a shell script is invoked with a set of command line parameters each of these parameters are copied into **special variables** that can be accessed.
- **\$0** This variable that contains the name of the script
- **\$1, \$2, \$n** 1st, 2nd 3rd command line parameter
- **\$#** Number of command line parameters
- **\$\$** process ID of the shell
- **\$@** same as **\$*** but as a list one at a time (see for loops later)
- **\$?** Return code 'exit code' of the last command
- **Shift** command: This shell command shifts the positional parameters by one towards the beginning and drops **\$1** from the list. After a **shift** **\$2** becomes **\$1** , and so on ... It is a useful command for processing the input parameters one at a time.



❖ SHELL SCRIPTING BASICS

❑ Shell Variables / *Example*

- `vi myinputs.sh`
`#!/bin/sh`
`echo Total number of inputs: $#`
`echo First input: $1`
`echo Second input: $2`
- `chmod u+x myinputs.sh`
- `myinputs.sh NSCS INFO CSE`
Total number of inputs: 3
First input: NSCS
Second input: INFO



❑ Shell Variable / Summary

Environment variables govern the behavior of programs in your Operating System.

Command	Description
VARIABLE_NAME= variable_value	Create a new variable
echo \$VARIABLE	To display value of a variable
env	Displays all environment variables
unset	Remove a variable
export Variable=value	To set value of an environment variable

- ❑ **Programming features of the UNIX/Linux shell**
 - Shell variables
 - **Operators**
 - Logic structures

The **Bash/Bourne/ksh** shell operators are divided into three groups:

1. **Defining and evaluating** operators,
2. **Arithmetic** operators, and
3. **Redirecting and piping** operators



❑ Programming features of the UNIX/Linux shell

■ Defining and Evaluating operators

A shell variable take on the generalized form **variable=value** (except in the C shell).

```
$ set x=37; echo $x
```

```
37
```

```
$ unset x; echo $x
```

```
x: Undefined variable.
```

You can set a pathname or a command to a variable or substitute to set the variable.

```
$ set mydir=`pwd`; echo $mydir
```

❑ Programming features of the UNIX/Linux shell

■ Arithmetic Operators

expr supports the following operators:

- Arithmetic operators: +, -, *, /, %
- Comparison operators: <, <=, ==, !=, >=, >
- Boolean/logical operators: &, |
- Parentheses: (,)
- Precedence is the same as C, java

```
■ vi math.sh
#!/bin/sh
count=5
count=`expr $count + 1`
echo $count
■ chmod u+x math.sh
■ math.sh
```

6

```
■ vi real.sh
#!/bin/sh
a=5.48
b=10.32
c=`echo "scale=2; $a + $b" |bc`
echo $c
■ chmod u+x real.sh
■ ./real.sh
```

15.80



❖ SHELL PROGRAMMING

❑ Programming features of the UNIX/Linux shell

■ Arithmetic Operators

var++ ,var-- , ++var , --var	post/pre-increment/decrement
+ , -	add subtract
* , / , %	multiply/divide, remainder
**	power of
! , ~	logical/bitwise negation
& ,	bitwise AND, OR
&&	logical AND, OR



❑ Programming features of the UNIX/Linux shell

- Shell variables
- Operators
- Logic structures

The four basic logic structures needed for program development are:

- **Sequential logic:** to execute commands in the order in which they appear in the program.
- **Decision logic:** to execute commands only if a certain condition is satisfied.
- **Looping logic:** to repeat a series of commands for a given number of times.
- **Case logic:** to replace “if then/else if/else” statements when making numerous comparisons.



❑ Programming features of the UNIX/Linux shell

■ Strings in Shell Scripting

Similar to all the programming languages **Bash** also has the **String** data type which indicates a set of characters. To denote inputs as **String** you must enclose it within the **double quotation(“”)**. Values passed as strings are considered as text rather than a number or variable. Therefore, **Bash** provides an additional set of operators for the **String** data type.

The syntax for Strings in Shell Scripting is given below :

```
STRING_NAME="STRING_VALUE"
```

The String operators in Shell Scripting are as follows:

String Operators		
< (Less than)	== (Equal)	+= (Concatenation)
> (Greater than)	!= (Not equal)	



❑ Programming features of the UNIX/Linux shell

■ Loops in Shell Scripting

Loops are introduced in programming languages to run tasks in a repetitive manner. It iterates a set of statements within a limit depending on conditions. **Bash Scripting** provides **three** types of loops for statement iterations. These are the **for loop**, the **while loop**, and the **until loop**. Syntaxes for each of the loops are listed below.

Syntaxes for Loops in Bash Scripting:

<pre>for item in item1 item2 ... itemN OR, for ((i=initial_val; i<=terminating_val; i++)) do #code to execute done</pre>	<pre>while [condition] do #code to execute done</pre>	<pre>until [condition] do #code to execute done</pre>
--	--	--



❑ Programming features of the UNIX/Linux shell

■ Functions in Shell Scripting

Functions are one of the popular concepts of programming languages. It is a piece of code that can be called and executed as many times as you want. Thus, functions offer efficiency, code optimization, and minimization.

Functions in **Bash** work in a similar way as functions in other programming languages. However, there are some rules and syntaxes that you must follow while using them in your script.

The syntax for Function in Shell Scripting:

```
FUNCTION_NAME () {  
#codes to execute  
}
```

OR:

```
FUNCTION_NAME () { #code to execute; }
```



❑ Programming features of the UNIX/Linux shell

■ Functions in Shell Scripting

The rules for Function in Shell Scripting are as follows :

- Functions must be defined before using/calling them.
- You may pass arguments to functions while calling them.
- To access arguments inside the function, use **\$1, \$2, \$3 ...** and so on according to the number and sequence of arguments passed.
- The scope of the variables declared inside a function remains within the function.



❖ SHELL PROGRAMMING

❑ Programming features of the UNIX/Linux shell

■ Functions in Shell Scripting

Example : Check if a String is a Palindrome

Write the code to check a palindrome inside the function “**Palindrome()**” and call it by passing the desired string:

Code:

```
#!/bin/bash
Palindrome () {
    s=$1
    if [ "$(echo $s | rev)" == "$str" ]
    then
        echo "The string is a Palindrome"
    else
        echo "The string is not a palindrome"
    fi
}
read -p "Enter a string: " str
Palindrome "$str"
```

Output:

```
Enter a string: wow
The string is a Palindrome
```



❑ Programming features of the UNIX/Linux shell

■ Task-Specific Bash Scripts

In addition to the **conceptual bash scripts**, in this section, you will find some **task-specific script examples**. These scripts are mostly related to the **regular process** that you run on your system.

Hence, explore the examples below to **get more hands-on experience with Shell Scripting ...**

- Regular Expression Based Scripts
- File Operations with Shell Scripts
- File Permission Based Shell Scripts
- Network Connection Based Shell Scripts
- Process Management Based Shell Scripts
- System Information Based Shell Scripts
- **Advanced Tasks with Shell Scripts :**
 - ✓ *Automated Backup*
 - ✓ *Generate Alert if disk space usage goes over a threshold*
 - ✓ *Create a new user and add to sudo group*
 - ✓ *Monitor Network Traffic*



❑ To Script or Not to Script

■ Pros

- File processing
- Glue together compelling, customized testing utilities
- Create powerful, tailor-made manufacturing tools
- Cross-platform support
- Custom testing and debugging

■ Cons

- Performance slowdown
- Accurate scientific computing



❑ Summary

- ☒ The best way to improve your shell scripting skills is to **practice regularly and work on projects that interest you.**

Further practice with scripting will improve automation skills.

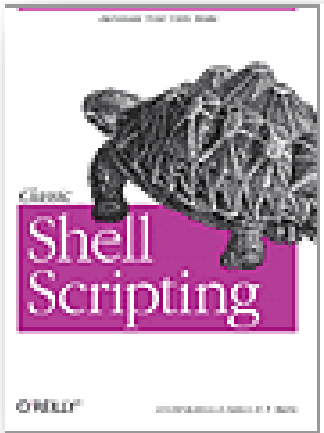
- ☒ Keep in mind that there are often **multiple ways to solve a shell scripting problem.**
- ☒ Remember to always **test your scripts thoroughly before using them** in a production environment.

From **complex task automation** to **efficient data manipulation**, you now possess the **ability to tackle real-world challenges** with confidence.

Embrace the power of advanced shell scripting and unlock a world of automation and efficiency.

❖ SHELL PROGRAMMING

❑ Reference Books



- **Class Shell Scripting**
<http://oreilly.com/catalog/9780596005955/>
- **LINUX Shell Scripting With Bash**
<http://ebooks.ebookmall.com/title/linux-shell-scripting-with-bash-burtch-ebooks.htm>
- **Shell Script in C Shell**
<http://www.grymoire.com/Unix/CshTop10.txt>
- **Linux Shell Scripting Tutorial**
<http://www.freeos.com/guides/lsst/>
- **Bash Shell Programming in Linux**
http://www.arachnoid.com/linux/shell_programming.html
- **Advanced Bash-Scripting Guide**
<http://tldp.org/LDP/abs/html/>
- **Unix Shell Programming**
<http://ebooks.ebookmall.com/title/unix-shell-programming-kochan-wood-ebooks.htm>



THANK YOU for your attention!



Questions ?



For more information about my research works, **Contact Information:**

Dr. Sassi BENTRAD

LISCO Laboratory : <http://lisco.univ-annaba.dz/>

☎ : +213 ...

✉ : sassi.bentrad.enscs@gmail.com // sassi.bentrad@enscs.edu.dz

in : www.linkedin.com/in/sassi-bentrad/

🏠 : <http://www.bentrad-sassi.sitew.com/>

ORCID
Connecting Research
and Researchers

iD : orcid.org/0000-0002-7458-8121

RESEARCHERID : [A-9442-2013](https://pubs.acs.org/doi/10.26434/chemrxiv-2013-9442)

SCOPUS Author ID : [44461052600](https://pubs.acs.org/doi/10.26434/chemrxiv-2013-9442)

